MENTORIA DE IA APLICADA · APOSTILA

Módulo 08 — Deploy e operação

O curador como Cloudflare Worker agendado: limites da plataforma, rastreio de custo e latência, versões do prompt com portão e rollback, evals em CI e um job que aguenta produção.

---

1. Visão geral
2. Conceitos
3. Exercício
4. Projeto final — etapa M8
5. Para revisar

Fonte: https://github.com/MateusCastro2203/Mentoria-AI

Visão geral

Objetivos de aprendizagem

Ao final, a pessoa consegue:

1. escolher onde rodar um fluxo com LLM e transformar os limites da plataforma (tempo, CPU, chamadas por execução) em decisões de desenho;
2. colocar um fluxo agendado num Cloudflare Worker com Cron Trigger e estado num KV, e testá-lo localmente;
3. rastrear cada chamada ao modelo (papel, item, tokens, latência) e calcular custo e latência (p50/p95) por execução;
4. versionar o prompt como um artefato, promover uma versão só se ela passar no portão de avaliação, e reverter sem novo deploy;
5. montar um CI com duas velocidades: testes offline em todo push e eval com o modelo de verdade quando o prompt muda;
6. escrever um job que aguenta produção: idempotência, retry só para erro transitório, backoff e teto de custo.

Mensagem central

> Em produção, o que importa é o que você consegue **ver** e **desfazer**. Rastreie cada chamada (onde vai o tempo e o dinheiro), versione o prompt como código (com um portão de avaliação antes e um botão de voltar depois) e escreva o job supondo que ele vai rodar duas vezes, falhar no meio e custar mais do que você pensou.

Conceitos

Até a Aula 7, o curador rodava no seu terminal, quando você mandava. Em produção, ele roda **sozinho**, toda segunda às 9h, num lugar que você não vê. Três perguntas passam a mandar no desenho:

1. **Onde roda, e com que limites?**
2. **Como eu sei o que aconteceu** (quanto custou, onde demorou, qual prompt foi usado)?
3. **Como eu desfaço** quando der errado?

---

1. Empacotar e escolher a plataforma

O curso usa três destinos possíveis: **Cloudflare Workers**, **Google Apps Script** e **Google Sites**. Para o curador, a escolha é o Workers:

Precisa de	Workers	Apps Script	Sites
Rodar TypeScript com as nossas bibliotecas (AI SDK, LangGraph, Zod)	sim (runtime JS, com compatibilidade Node via `nodejs_compat`)	não (outro runtime, sem npm)	não (páginas estáticas)
Rodar agendado	sim (Cron Triggers)	sim (gatilhos de tempo)	não
Guardar estado entre execuções	sim (KV, D1, Durable Objects)	sim (planilhas, propriedades)	não

Apps Script e Sites continuam ótimos para outra coisa: uma página com a última edição, um formulário que chama o Worker. A regra geral: **escolha pelo runtime e pelo estado de que você precisa**, não pela familiaridade.

Empacotar

No Workers não há disco. Tudo o que o código lê precisa entrar no pacote (o *bundle*) que o `wrangler` monta: os feeds RSS e a skill entram como texto (`rules` com tipo `Text` no `wrangler.jsonc`), a lista de fontes entra como JSON. O que muda de ambiente para ambiente (endereço do modelo, preço, limites) vira **variável**; o que é secreto (token do editor, chave do provedor) vira **segredo** (`wrangler secret put`; no local, `.dev.vars`, fora do git).

Dois ajustes que só aparecem quando você tenta de verdade (e que o curso já fez):

- **AsyncLocalStorage do LangGraph.** O `wrangler` empacota a versão "navegador" do `@langchain/langgraph`, que não liga o AsyncLocalStorage. Sem ele, `interrupt()` falha com *"Called interrupt() outside the context of a graph"*. A correção está em `projeto-final/worker/recursos.ts` (três linhas).
- **Caminho de arquivo no carregamento do módulo.** `new URL("../skills/…", import.meta.url)` funciona no Node, mas no Workers `import.meta.url` não é um caminho `file:`, e o Worker nem sobe. Calcule caminhos só quando for usar, ou proteja (veja `PASTA_SKILL` em `src/m04/skill.ts`).

Os limites viram requisito

Toda plataforma tem limites, e eles mudam o desenho. Os do Workers que pesam para nós (consulte a página de limites, porque eles mudam):

Limite	Plano gratuito	Plano pago	O que muda no curador
Subrequests (cada `fetch`, incluindo cada chamada ao modelo)	50 por execução	10.000	32 notícias geram ~70 chamadas: não cabe. `MAX_NOTICIAS=8` cabe (≤ 40).
CPU por execução de cron	10 ms	30 s (intervalo < 1 h) a 15 min	Esperar o modelo **não** conta; montar o grafo e o JSON contam.
Duração (relógio) de um cron	15 min	15 min	Uma edição com 32 notícias num modelo local passa disso.
Escrita no KV	1 por segundo na mesma chave	igual	O checkpoint é gravado **uma vez**, no fim, e não a cada passo do grafo.

Por isso, na versão agendada, o **coletor virou código**: pega as N primeiras notícias dos feeds, sem agente. Às 9h de segunda ninguém está olhando a trajetória de um agente, e cada passo dele seria uma chamada a mais no limite. É a pergunta da Aula 4 de novo: agente só onde a autonomia se paga.

---

2. O curador como Worker

```
Cron (seg 9h) ──► scheduled() ──► grafo da M7 (checkpoint no KV) ──► pausa? ──►
registra execução + rastro
                                                                       │
HTTP (editor) ──► GET /pendentes · POST /retomar ──► Command({ resume }) ──► publicar
no KV
HTTP (operação) ──► GET /execucoes · GET /prompts · POST /prompts · POST
/prompts/reverter
```

- **O grafo é o mesmo da Aula 7.** Mudam os papéis (coletor em código) e o checkpointer (`CheckpointEmKV`, o MemorySaver gravado no KV).
- **A "tela" da revisão virou HTTP.** O terminal da Aula 7 chamava `Command({ resume })`; agora quem chama é o `POST /retomar`. O grafo não percebeu a diferença, como prometido.
- **Rotas que mudam algo pedem token** (`Authorization: Bearer …`). Uma rota de aprovação aberta na internet é uma porta para qualquer um publicar.
- **Uma execução por dia** (idempotência): se o cron disparar de novo no mesmo dia, o Worker registra "pulada" e não faz nada.

Estado durável: KV, D1, Durable Objects

O KV é simples e serve para a aula, mas tem dois limites para um checkpointer: **consistência eventual** (uma escrita pode levar 60 s ou mais para aparecer em outros pontos da rede) e **nenhum controle de concorrência** (dois `POST /retomar` ao mesmo tempo: um apaga o outro). Em produção:

- **D1** (SQLite gerenciado) ou um **Durable Object com SQLite**: gravam a cada passo e serializam o acesso. A própria documentação do LangChain para Workers usa um Durable Object por thread e avisa que o `MemorySaver` não é durável entre instâncias.
- Um checkpointer de banco do ecossistema LangGraph (SQLite, Postgres) se o grafo rodar fora do Workers.

---

3. Rastreio: o que aconteceu, quanto custou, onde demorou

Um **rastro** (*trace*) é o registro de uma execução, quebrado em partes (*spans*): cada chamada ao modelo, com início, duração e atributos. No curso, ele nasce no único lugar por onde toda chamada passa, o `@mentoria/llm`:

```
const parar = observarChamadas((evento) => eventos.push(evento));       // assina
await comAtributos({ papel: "redigir", alvo: "n13" }, () => redigir()); // dá contexto
// evento: { operacao, modelo, latenciaMs, tokensEntrada, tokensSaida, erro?,
atributos: { papel, alvo, execucao, prompt } }
```

O contexto atravessa os `await` com **AsyncLocalStorage** (Node e Workers), então 8 classificações em paralelo não se misturam.

Custo

`custo = tokens de entrada × preço de entrada + tokens de saída × preço de saída` (preços por milhão de tokens, na página do seu provedor). Com modelo local, o preço é zero, mas o custo vira **tempo e máquina**, e os tokens continuam sendo a medida.

Duas regras:

- **Estimar é para teto, medir é para fatura.** `caracteres ÷ 4` dá uma ordem de grandeza antes de chamar (o exercício usa isso como orçamento); depois, use o `usage` que o provedor devolve.
- **O número real surpreende.** No nosso rastro, o **revisor** gerou 87% dos tokens de saída. Com saída estruturada (JSON schema), o Ollama contou 758 tokens para uma resposta de 326 caracteres; sem schema, 152 tokens para 538 caracteres. Sem rastro, ninguém olharia para o revisor.

Latência: p50 e p95

- **p50** (mediana): metade das chamadas foi mais rápida que isso. É a experiência típica.
- **p95**: 95% foram mais rápidas. É a experiência dos casos ruins, e é ela que estoura o tempo limite.
- **A soma das latências não é a duração.** Com chamadas em paralelo, a soma passa da duração (no nosso caso, 829 s de latência somada numa execução de 245 s). Em modelo local, as chamadas paralelas esperam na fila e o p95 sobe.

Padrões de mercado

- **OpenTelemetry** é o padrão aberto para rastros, e há convenções semânticas para GenAI (nomes de atributos como os de tokens de entrada e saída). O AI SDK emite spans nesse formato (pacote `@ai-sdk/otel`).
- **Workers Logs** guarda o que o Worker escreve no console; por isso o curador escreve **uma linha JSON por execução** (fácil de filtrar). O Workers também tem rastreamento automático (em beta) e exportação para ferramentas de observabilidade via OpenTelemetry.

O nosso rastreador caseiro existe para você ver as peças e testar offline. Em produção, prefira o padrão.

---

4. Versionar o prompt (e o modelo) e voltar atrás

O prompt é **código que muda o comportamento** do sistema, então precisa do mesmo cuidado:

1. **Versão imutável com hash.** `v1` sempre aponta para o mesmo texto; mudou o texto, é outra versão. O hash amarra a avaliação ao texto avaliado.
2. **Portão de avaliação antes de ativar**, em duas camadas:
   - **absoluto** (no CI, `m08:avaliar`): macro-F1 ≥ 0,8 e nenhum caso especial publicado;
   - **relativo** (no Worker, na hora de promover): não pode ser pior que a versão **no ar**, avaliada com o mesmo modelo. Só o Worker sabe qual versão está no ar.
3. **Registro de qual versão rodou** em cada execução (`prompt: { id, hash }`): sem isso, uma edição ruim não tem culpado.

4. **Rollback sem deploy**: a versão ativa é um dado no KV; voltar é trocar um ponteiro. Rollback não passa pelo portão: é a saída de emergência, para uma versão que já esteve no ar.

O **modelo** é parte da versão: trocar `LLM_MODEL` muda tudo. Por isso a avaliação guarda o modelo e o portão relativo só compara avaliações do mesmo modelo.

Rollback do código

O Workers guarda as versões publicadas: `wrangler versions list`, `wrangler rollback [versão]`, e `wrangler versions deploy` com porcentagens para uma **implantação gradual** (ex.: 10% na nova). Duas ressalvas da documentação: o rollback alcança as últimas 100 versões e **não desfaz dados** (KV, D1, filas). Nada desfaz um e-mail enviado ou uma edição publicada.

Canário

Implantação gradual aplicada ao prompt: uma parte das execuções usa a versão nova, escolhida de forma **determinística** (o mesmo dia ou o mesmo cliente cai sempre na mesma versão). Com dados suficientes, promove ou reverte. É o desafio do exercício.

---

5. Evals em CI: duas velocidades

	Testes offline	Eval com modelo
Quando	todo push e PR	quando o prompt, a eval ou o dataset mudam (e manual)
Modelo	simulado (`MockLanguageModelV4`)	o de verdade (Ollama no runner)
Tempo	segundos	minutos (CPU do runner)
Pega	código quebrado, contrato, regras	regressão de qualidade do prompt

Os dois workflows estão em `.github/workflows/`: `ci.yml` (typecheck + testes) e `eval-prompt.yml` (instala o Ollama, baixa o modelo e roda `m08:avaliar`). Reprovou no portão, o PR fica vermelho. O promptfoo também tem uma action oficial que comenta o resultado no PR.

Por que não rodar a eval com modelo em todo push? Ela é lenta, custa, e um sinal que demora e falha por ruído vira um sinal que o time ignora.

---

6. Um job que aguenta produção

O exercício trata das quatro propriedades que todo job agendado com LLM precisa ter:

1. **Idempotência.** O cron pode disparar duas vezes (reinício, falha de rede, alguém clicou em "executar"). Use uma chave por unidade de trabalho (o dia) e não refaça o que já deu certo.
2. **Retry só para o que é transitório.** Limite de taxa (429), erro do servidor (5xx), rede: vale tentar de novo. Resposta fora do schema, prompt errado, bug: não vai melhorar, e cada tentativa custa.
3. **Backoff exponencial (com jitter).** Espere 1 s, 2 s, 4 s… para não martelar um serviço que já está sofrendo. *Jitter* (um pouco de aleatoriedade na espera) evita que mil clientes tentem de novo no mesmo instante.
4. **Retry numa camada só.** O artigo da Amazon Builders' Library mostra o efeito de cada camada tentar de novo por conta própria: no exemplo deles, a carga no banco cresce 243 vezes. O AI SDK já tenta 2 vezes sozinho; se o seu código também tenta, desligue um dos dois (`retentativas: 0` no `@mentoria/llm`).

E um quinto, específico de LLM: **teto de custo**. Uma entrada inesperadamente grande (alguém colou um log de 2 MB num chamado) não pode virar uma fatura inesperada.

---

7. Monitoramento em produção (leitura extra)

Cortado da aula por tempo. O que observar depois que está no ar:

- **Os quatro sinais de ouro** do livro de SRE do Google: **latência, tráfego, erros e saturação**. Para um job com LLM: duração e p95 por papel; execuções e itens por dia; taxa de falha, de retry e de execuções "puladas"; uso dos limites (subrequests, CPU, cota do provedor).
- **Sinais de qualidade**, que não aparecem nos quatro: taxa de itens que vão para revisão humana, taxa de override (Aula 7), distribuição de confiança, tamanho dos resumos. Uma mudança brusca indica que algo mudou.
- **Drift.** O mundo muda (novos assuntos, novas fontes), o provedor atualiza o modelo e o comportamento muda junto. Sculley et al. (2015) chamam isso de dívida técnica escondida dos sistemas de ML: o sistema depende de dados e de um modelo que mudam por fora do código. Antídoto: re-rodar a eval periodicamente, não só quando o prompt muda, e fixar a versão do modelo quando o provedor permitir.
- **Avaliação online.** As decisões humanas da Aula 7 viram dados rotulados de graça: guarde-as e use-as para atualizar o dataset de eval.
- **Alertas que alguém lê.** Alerta para o que exige ação (o cron não rodou, a taxa de falha passou de X%), não para tudo. Um alerta que dispara toda semana vira ruído.

Referências

- Cloudflare Workers — Cron Triggers: https://developers.cloudflare.com/workers/configuration/cron-triggers/
- Cloudflare Workers — limites: https://developers.cloudflare.com/workers/platform/limits/

- Cloudflare Workers — compatibilidade com Node.js e AsyncLocalStorage: https://developers.cloudflare.com/workers/runtime-apis/nodejs/ · https://developers.cloudflare.com/workers/runtime-apis/nodejs/asynclocalstorage/
- Workers KV — limites: https://developers.cloudflare.com/kv/platform/limits/ · como funciona (consistência): https://developers.cloudflare.com/kv/concepts/how-kv-works/
- Workers — rollbacks: https://developers.cloudflare.com/workers/configuration/versions-and-deployments/rollbacks/ · implantação gradual: https://developers.cloudflare.com/workers/configuration/versions-and-deployments/gradual-deployments/
- Workers — logs: https://developers.cloudflare.com/workers/observability/logs/workers-logs/ · traces: https://developers.cloudflare.com/workers/observability/traces/ · exportar OpenTelemetry: https://developers.cloudflare.com/workers/observability/exporting-opentelemetry-data/
- LangChain — deploy em Cloudflare Workers: https://docs.langchain.com/langsmith/deploy-cloudflare-workers
- promptfoo — CI/CD: https://www.promptfoo.dev/docs/integrations/ci-cd/ · GitHub Action: https://www.promptfoo.dev/docs/integrations/github-action/
- AI SDK — telemetria: https://ai-sdk.dev/docs/ai-sdk-core/telemetry
- OpenTelemetry — convenções semânticas para GenAI: https://github.com/open-telemetry/semantic-conventions-genai
- Ewaschuk, R. (2016). *Monitoring Distributed Systems*. Em: Beyer, B. et al. (orgs.), *Site Reliability Engineering*. O'Reilly. https://sre.google/sre-book/monitoring-distributed-systems/
- Amazon Builders' Library. *Timeouts, retries, and backoff with jitter*. https://builder.aws.com/content/3EumjoZascWd1oZiEgL8ORlv3qE/timeouts-retries-and-backoff-with-jitter
- Sculley, D. et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 28. https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

Exercício

> Tarefa assíncrona. Os testes rodam offline: não precisa de modelo nem de conta na Cloudflare.

Todo dia às 8h, um Cloudflare Worker resume os chamados de suporte abertos no dia (dados sintéticos em `dados/chamados.json`) com um LLM e guarda o resultado num KV. Parece simples, mas em produção:

- o cron às vezes **dispara duas vezes**;
- o provedor devolve **429** (limite de taxa) ou **503** (fora do ar) de vez em quando;
- às vezes o modelo devolve **lixo**, e tentar de novo não adianta;
- um chamado com um log colado pode fazer a chamada custar **100 vezes mais**.

```
cron ─► executarDia(data)
          ├─ já deu "ok" hoje? ──────────────► devolve o registro (não chama o modelo)
          ├─ nenhum chamado no dia? ─────────► "sem-chamados"
          ├─ custo estimado > orçamento? ────► "orcamento"
          └─ resumir ─┬─ ok ──────────────────► "ok" (tokens, custo, resumo)
                      ├─ ErroTransitorio ─────► espera 1 s, 2 s… e tenta de novo (até
maxTentativas)
                      └─ outro erro ──────────► "falhou" (sem insistir)
```

Como rodar

Da raiz do repositório:

```
pnpm -F @mentoria/ex08-operacao test              # versão base
pnpm -F @mentoria/ex08-operacao test:desafio      # desafio extra
pnpm -F @mentoria/ex08-operacao demo:falhas       # offline: as cenas de produção com
o seu código
pnpm -F @mentoria/ex08-operacao dev               # o Worker de verdade, local
(precisa do Ollama)
```

Os testes **falham** até você implementar. A solução de referência está em `solucao/`. Tente antes de abrir. `pnpm -F @mentoria/ex08-operacao test:solucao` roda os mesmos testes contra ela.

Versão base

Arquivo	O que fazer
`src/custo.ts`	`estimarTokens` (regra de bolso: caracteres ÷ 4) e `custo` (tokens × preço por milhão)
`src/execucao.ts`	`executarDia`: idempotência, orçamento, retry com backoff só para erro transitório, registro no KV

Fornecidos: `src/estado.ts` (formatos, `ErroTransitorio`, `POLITICA_PADRAO`) e `src/worker.ts` (o Worker, que usa a sua `executarDia`).

Dicas:

- Tudo o que vem de fora (KV, modelo, relógio, espera) chega em `deps`. Por isso o teste roda em milissegundos: a "espera" de 2 s só é anotada.
- Idempotência é **antes** de tudo: a primeira coisa é olhar se o dia já deu certo.
- Para saber se vale tentar de novo: `erro instanceof ErroTransitorio`. O Worker transforma os erros do provedor (429, 5xx, rede) em `ErroTransitorio` para você.
- Espera da tentativa n+1: `esperaInicialMs × 2^(n−1)` → 1 s, 2 s, 4 s…

Feito = `pnpm -F @mentoria/ex08-operacao test` com todos os testes passando.

Ver no Worker (opcional, mas recomendado)

Com o Ollama rodando:

```
pnpm -F @mentoria/ex08-operacao dev -- --var SIMULAR_FALHAS:2      # as 2 primeiras chamadas falham (503 simulado)
curl "http://localhost:8787/cdn-cgi/local/scheduled?cron=0+8+*+*+*&time=1791878400000"
# cron de 13/10/2026
curl "http://localhost:8787/resumo?data=2026-10-13"
```

No log do `wrangler dev`, procure a linha `{"evento":"resumo-diario",…}` com `"tentativas":3`. Dispare o cron de novo e veja que nada muda. Com a solução, rodei: 4 chamados, 3 tentativas (2 falhas simuladas), 215 tokens de entrada e 71 de saída, ~8 s (3 s são as esperas).

Repare em `retentativas: 0` em `src/worker.ts`: o AI SDK já tenta 2 vezes sozinho. Com o seu retry por cima, seriam até 9 chamadas por execução.

Desafio extra — canário

Uma versão nova do prompt não vai para 100% das execuções de uma vez. Em `src/desafio/canario.ts`:

1. `escolherVersao`: uma parte das execuções (o `percentual`) vai para a candidata, escolhida de forma **determinística** pela chave (use `balde(chave)`, de 0 a 99);
2. `decidirCanario`: com dados suficientes da candidata, **promover** ou **reverter** comparando a taxa de falha dela com a da versão estável.

Feito = `pnpm -F @mentoria/ex08-operacao test:desafio` com todos os testes passando.

Ainda mais (opcional, precisa de conta na Cloudflare): publique o Worker (`wrangler kv namespace create RESUMOS`, troque o `id` no `wrangler.jsonc` e rode `wrangler deploy`). Ele vai precisar de um endpoint de modelo acessível pela internet (qualquer um compatível com a API da OpenAI; configure `LLM_BASE_URL` e `LLM_MODEL` como variáveis e a chave com `wrangler secret put LLM_API_KEY`).

Perguntas para pensar

1. Por que a idempotência olha só para o status "ok"? O que aconteceria se um dia que deu "orcamento" nunca mais rodasse?
2. Um erro 400 (pedido inválido) é transitório? E um timeout?
3. O orçamento usa uma **estimativa**. O que acontece se ela errar para baixo? Que outro limite você poria?
4. No canário, por que a escolha precisa ser determinística? O que daria errado se fosse aleatória a cada execução?

Projeto final — etapa M8

Etapa M8 — o curador em produção (Cloudflare Workers)

Objetivo: o curador roda sozinho, toda segunda às 9h, num Cloudflare Worker. A revisão humana da M7 vira HTTP, cada chamada ao modelo entra num rastro (custo, tokens, latência por papel), o prompt ganha versões com portão de avaliação e rollback, e o CI testa tudo a cada push.

```
Cron ─► scheduled() ─► grafo da M7 (checkpoint no KV) ─► pausa ─► GET /pendentes · POST
/retomar ─► publica no KV
                          │
                          └─ rastro: { papel, item, tokens, latência } ─► GET
/execucoes (custo, p50, p95)
m08:avaliar (CI) ─► avaliação com hash ─► POST /prompts (portão contra a versão ativa)
─► POST /prompts/reverter
```

Implemente em `src/m08/`:

1. **`rastreio.ts`:**
   - `percentil` (posição mais próxima);
   - `custoDaChamada` (tokens × preço por milhão);
   - `resumirRastro` (total e por papel: chamadas, erros, tokens, custo, p50, p95);
   - `rastrearPapeis` (cada papel roda dentro de `comAtributos({ papel, alvo })`).
2. **`prompts.ts`:**
   - `registrarVersao` (imutável, com hash);
   - `versaoAtiva`;
   - `promover` (portão: hash igual, nenhum caso especial, macro-F1 mínimo e não pior que a versão ativa com o mesmo modelo);
   - `reverter` (rollback pelo histórico).

Fornecidos:

- `nuvem.ts`: o Worker (`scheduled` + rotas HTTP), com os papéis da versão agendada (o coletor vira código);
- `checkpoint-kv.ts`: o checkpointer no KV;
- `armazenamento.ts`, `tipos-rastreio.ts` e `tipos-prompts.ts`;
- `worker/`: as entradas do Worker, com o seu código e com a solução;
- `wrangler.jsonc`;
- `scripts/m08-avaliar.ts`: a eval de uma versão da skill;
- `scripts/m08-operar.ts`: o cliente do Worker;
- `.github/workflows/`: o CI e a eval do prompt.

O `@mentoria/llm` ganhou `observarChamadas` e `comAtributos`: cada chamada ao modelo emite um evento.

Feito =

1. `pnpm -F @mentoria/curador exec vitest run test/m08.test.ts` verde (offline);
2. uma edição gerada pelo cron no Worker local, revisada por HTTP e publicada:

```
cp projeto-final/.dev.vars.example projeto-final/.dev.vars    # o token do editor (local)
pnpm -F @mentoria/curador m08:dev                              # terminal 1: o Worker (porta 8787)
pnpm -F @mentoria/curador m08:operar rodar                     # terminal 2: dispara o cron (~4 min, 8 notícias)
pnpm -F @mentoria/curador m08:operar pendentes
pnpm -F @mentoria/curador m08:operar responder '{"<id>":{"acao":"aprovar"}}'
pnpm -F @mentoria/curador m08:operar execucoes                 # custo, tokens e latência por papel
```

3. uma versão do **seu** prompt avaliada e promovida, e um rollback:

```
pnpm -F @mentoria/curador m08:avaliar                          # avalia skills/curar-noticia → saidas/m08-avaliacao.json
pnpm -F @mentoria/curador m08:operar promover v1
# copie a pasta da skill, mude o SKILL.md, avalie com -- --pasta <cópia>/curar-noticia e promova como v2
pnpm -F @mentoria/curador m08:operar reverter
```

O worker usa o **seu** código das etapas M2 a M7 (`classificarTipado`, `decidirPublicacao`, `montarPromptDaSkill`, `montarMarkdown`, `montarGrafoHitl`). Se travou numa delas, `m08:dev:solucao` usa as soluções.

Responda (para a discussão final):

- Qual papel custou mais tokens? E qual teve o pior p95? Você esperava?
- A sua mudança de prompt passou no portão? Se passou no CI e foi barrada no Worker, por quê?
- O que o rollback do prompt **não** desfaz?

Desafio extra:

- Publique de verdade (`wrangler kv namespace create CURADOR`, troque o `id`, `wrangler secret put EDITOR_TOKEN`, `wrangler deploy`). O Worker vai precisar de um endpoint de modelo acessível pela internet: qualquer um compatível com a API da OpenAI, configurado

em `LLM_BASE_URL`, `LLM_MODEL` e `wrangler secret put LLM_API_KEY`. Meça o tempo de CPU no painel.

- Troque o `CheckpointEmKV` por um checkpointer que grave a cada passo (D1 ou Durable Object com SQLite).
- Exporte o rastro em OpenTelemetry (`@ai-sdk/otel`) para uma ferramenta de observabilidade.

Para revisar

Perguntas para discussão (15 min)

1. No seu trabalho, qual job com IA rodaria agendado? O que acontece se ele rodar duas vezes? E se não rodar?
2. Quem decide promover uma versão de prompt: o CI, uma pessoa, os dois? Que número você exigiria?
3. O revisor custa 87% dos tokens de saída. Você o tiraria, trocaria por um modelo menor ou por regras? Como saberia se a qualidade caiu?
4. Rollback do prompt é instantâneo; o da edição já enviada, não. O que mais no seu sistema não volta com rollback?
5. **Retrospectiva:** das oito aulas, o que você já usaria amanhã? O que ficou faltando?

Armadilhas comuns

- **Job não idempotente:** o cron dispara duas vezes (acontece) e o e-mail sai duas vezes.
- **Retry em tudo:** tentar de novo um erro de schema não melhora nada e gasta tokens. Retry só para erro transitório (limite de taxa, 5xx, rede).
- **Retry em duas camadas:** o SDK tenta 3 vezes, seu código tenta 3 vezes → até 9 chamadas. Escolha uma camada.
- **Custo estimado como fatura:** `caracteres ÷ 4` serve de teto; a fatura vem do `usage` que o provedor devolve.
- **Somar latências e chamar de duração:** com paralelismo, a soma passa (muito) da duração. Olhe p50 e p95 por papel.
- **Prompt sem versão no registro da execução:** sem isso, não dá para saber qual prompt gerou a edição ruim.
- **Avaliação de um texto, promoção de outro:** confira pelo hash.
- **Achar que rollback desfaz tudo:** o código e o prompt voltam; dados gravados, mensagens enviadas e edições publicadas não.
- **Eval com LLM em todo push:** lenta e cara; o time passa a ignorar. Rode quando o prompt, o modelo ou o dataset mudam.
- **Gravar no KV a cada passo do grafo:** o KV aceita 1 escrita por segundo na mesma chave. Grave no fim (ou use um banco).
- **Segredo no código ou no `wrangler.jsonc`:** use `wrangler secret put` (e `.dev.vars` no local, fora do git).